

ЗФедеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

Вариант №18765

Выполнил

Макогон Ярослав Вадимович

Номер группы: Р3218

Проверила

Бострикова Д. К.

Содержание

Цели	3
Задание.....	4
Решение.....	6
Доп	7
Выводы	9

Цели

- Разработать JSF-приложение по заданию
- Узнать, как работать с PrimeFaces
- Разобраться с использованием бинов (ManagedBean, CDI)
- Познакомится с ORM (Hibernate)

Задание

Разработать приложение на базе JavaServer Faces Framework, которое осуществляет проверку попадания точки в заданную область на координатной плоскости.

Приложение должно включать в себя 2 facelets-шаблона - стартовую страницу и основную страницу приложения, а также набор управляемых бинов (managed beans), реализующих логику на стороне сервера.

Стартовая страница должна содержать следующие элементы:

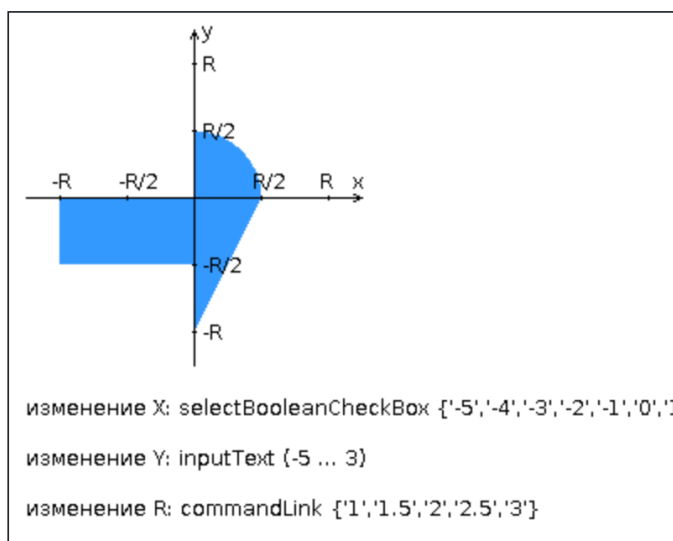
- "Шапку", содержащую ФИО студента, номер группы и номер варианта.
- Интерактивные часы, показывающие текущие дату и время, обновляющиеся раз в 13 секунд.
- Ссылку, позволяющую перейти на основную страницу приложения.

Основная страница приложения должна содержать следующие элементы:

- Набор компонентов для задания координат точки и радиуса области в соответствии с вариантом задания. Может потребоваться использование дополнительных библиотек компонентов - [ICEfaces](#) (префикс "ace") и [PrimeFaces](#) (префикс "p"). Если компонент допускает ввод заведомо некорректных данных (таких, например, как буквы в координатах точки или отрицательный радиус), то приложение должно осуществлять их валидацию.
- Динамически обновляемую картинку, изображающую область на координатной плоскости в соответствии с номером варианта и точки, координаты которых были заданы пользователем. Клик по картинке должен инициировать сценарий, осуществляющий определение координат новой точки и отправку их на сервер для проверки её попадания в область. Цвет точек должен зависеть от факта попадания / непадения в область. Смена радиуса также должна инициировать перерисовку картинки.
- Таблицу со списком результатов предыдущих проверок.
- Ссылку, позволяющую вернуться на стартовую страницу.

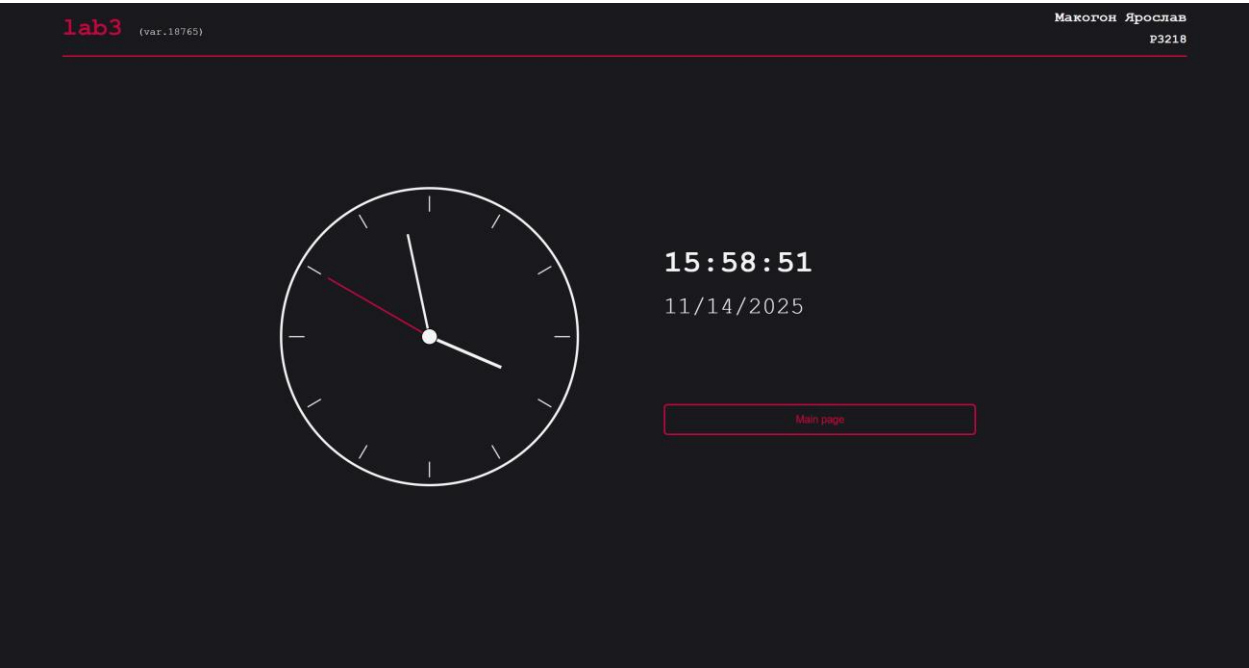
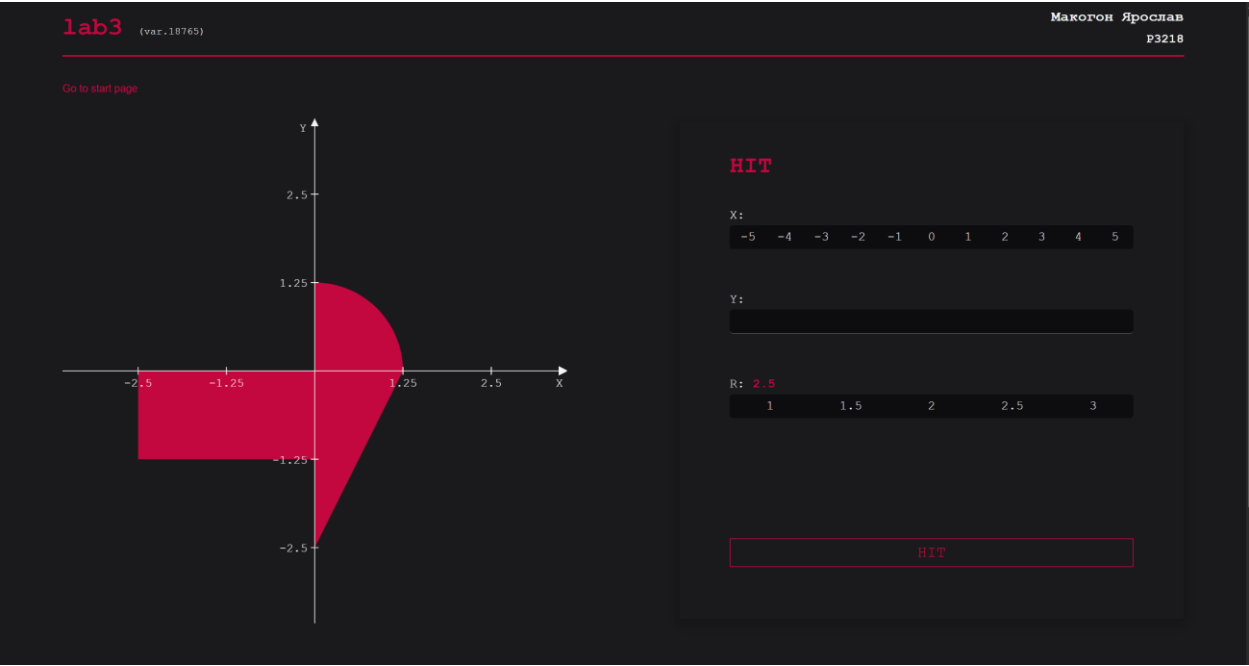
Дополнительные требования к приложению:

- Все результаты проверки должны сохраняться в базе данных под управлением СУБД Oracle.
- Для доступа к БД необходимо использовать JPA с ORM-провайдером на усмотрение студента.
- Для управления списком результатов должен использоваться Session-scoped Managed Bean.
- Конфигурация управляемых бинов должна быть задана с помощью аннотаций.
- Правила навигации между страницами приложения должны быть заданы в отдельном конфигурационном файле.



Решение

Код лабораторной работы на GitHub: <https://github.com/Not-N0w/web-lab3>



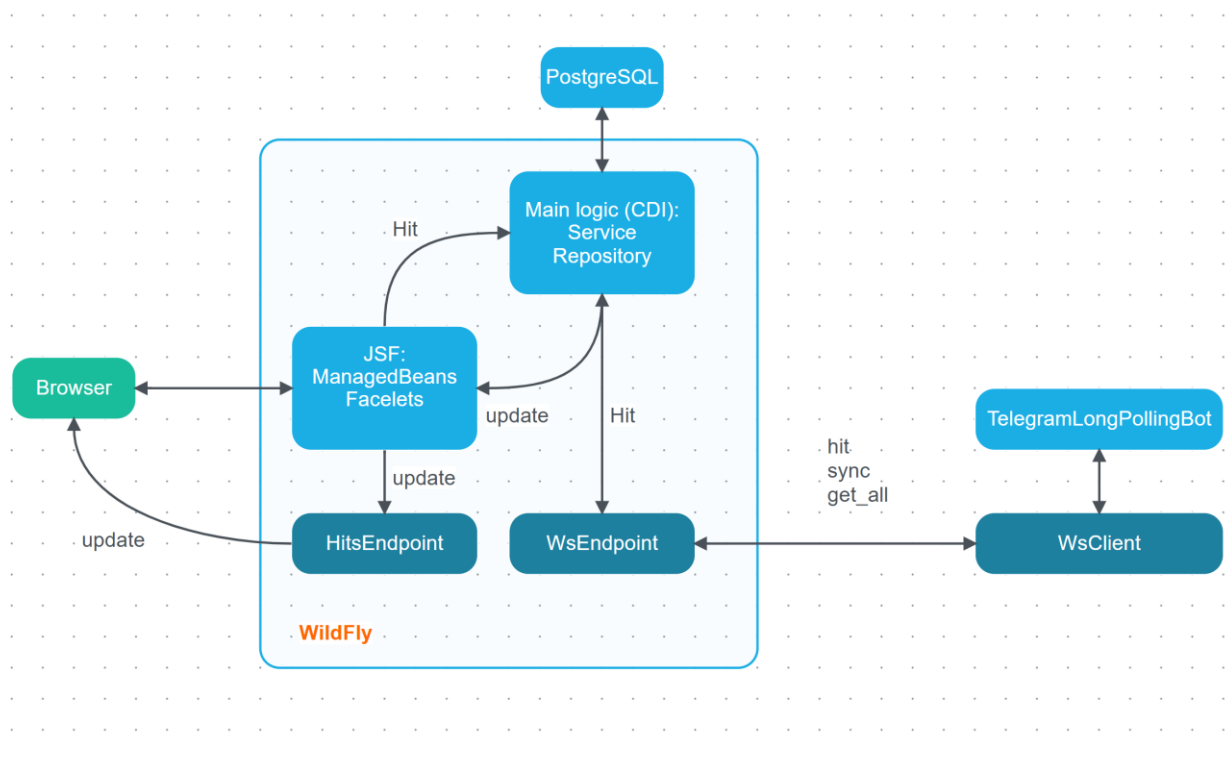
Доп

Код на GitHub: <https://github.com/Not-N0w/web-lab3>

Разработать Telegram-бота на Java, который:

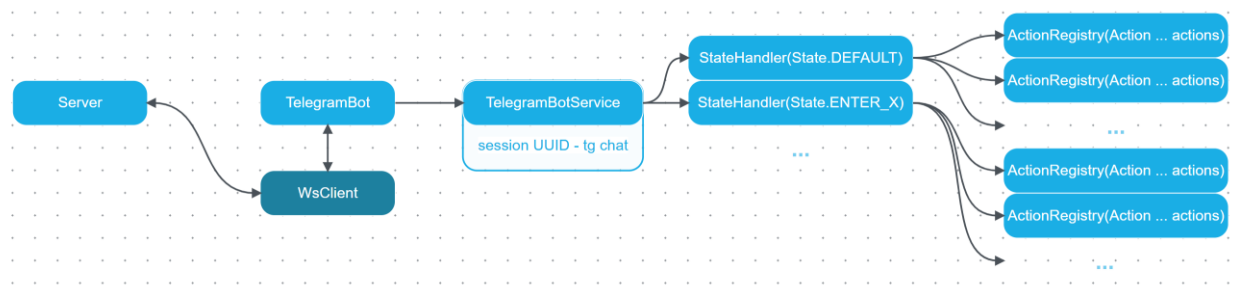
1. Принимает команды из Telegram и отправляет hit-запросы на сервер.
2. Умеет привязываться к сессии браузера.
3. Получает события из браузера (попадания) через WebSocket.
4. В реальном времени отправляет уведомления в Telegram о событиях в браузере и результатах запросов.
5. Обновление результатов в браузере при отправке запросов из бота.

Общая архитектура:



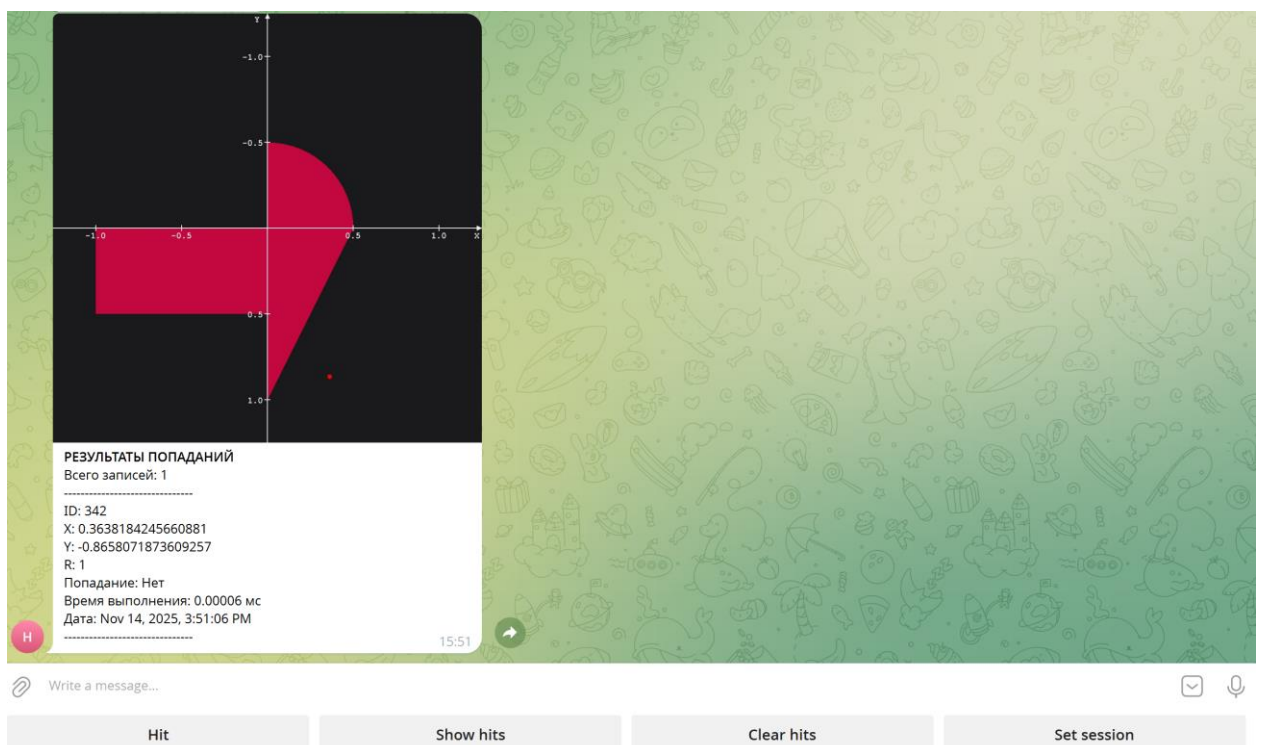
Все же было принято решение часть бинов вынести в CDI, во-первых, исходя из архитектурной логики (работа с бд и логика обработки попадания не относится к JSF), во-вторых, websocket живет вне контейнера managed бинов, поэтому инжектировать websocket в них затруднительно, так как его жизненным циклом тоже управляет контейнер + тяжело учитывать сессии. С CDI бинами же механизм связи с websocket +- очевиден.

Логика сессий. Есть сессия в браузере, с которой связаны session scoped managed beans из jsf. Для каждой сессии генерируем UUID. В слое Main logic храним сопоставления session scoped managed бина с UUID сессии. Также добавляем этот UUID к каждому хиту в бд. Таким образом, образуется что-то вроде области видимости хитов для каждой сессии. При "/start" в телеграм боте тоже генерируется UUID, который прикрепляется к текущему чату и отправляется с каждым запросом (тоже своя область видимости).



Telegram-бот – LongPollingBot. Хотелось, сделать красивое заполнение запроса (типо форма) с кнопками итд. К сожалению, в LongPollingBot нет логики для состояния чата, а это необходимо для реализации задуманного. Реализованная логика представлена на картинке.

Синхронизация. Пользователь может на стороне телеграм бота совершить синхронизацию с сессией из браузера. Для этого из браузера берется UUID, вводится в чат. На стороне тг, чат теперь имеет новый session id. После этого идет запрос на синхронизацию, который меняет в бд session id со старого на новый. Затем, совершается update (подгрузка новых данных в jsf) при помощи HitsEndpoint (websocket). Теперь при отправки данных в тг, они будут отображаться на странице, а при отправке данных на странице будет прилетать уведомление в тг.



Выводы

- Разработано jsf-приложение по зданию с использованием PrimeFaces
- Познакомился с Hibernate
- Разработан телеграм бот с синхронизацией данных с jsf.